

Video Transcript

Video 1: Introduction (04:51)

So, in this section, we're going to look at two of the most important components of web computing. We're going to look at the browser side, and we're going to look at the web server. Now, they use what's called request response computing. Usually, the client will make a request to the web server and the web server will respond with a message such as a web page, or containing data. The basics of client-server architecture will be investigated, and we'll look at HTTP as a universal messaging language. So, it mediates between applications written in Python, for example, and those written in JavaScript or Go or .Net.

We'll be looking at the Flask templating engine, which is called Jinja, and examining some of the abilities of that to combine Python into HTML pages. We'll be looking at Flask routes in some detail. These routes allow a program to call a specific route, which fires then a specific function on the web server. So, the client, by issuing the right kind of message, can fire a remote function. Now, this is the basics of distributed computing. Once, you can fire a function on a remote machine and get a response from it, then you're doing distributed computing. So, it's critical that you understand roots.

Now, we'll also look at the stateless concept of a web server. Web servers usually don't maintain state. If they had to remember everything they said to you, then it would get overwhelmed. So, basically, web servers are stateless. Now this means that when you message them, you must have some means of telling them who you are, and that's done through cookies. So, we're going to be looking in some detail at cookies and specially session cookies, and we're going to be looking at how those can be used to protect API routes, so application program interfaces. And we'll be seeing how Swagger can be used to detail an API, so that it can expose itself to others. So, if we're hitting some API, we need to know exactly, what arguments does it need, what's the structure, etc. and Swagger can tell us that. So, we'll be looking in some detail at developing a Swagger interface to our demonstration program.

Okay, so this is the fun part. Let's go and write our Flask web server. One of the things that you'll find probably most trying is dealing with security tokens. In order to operate on the web in the

cloud, you have to generate security tokens so that your message will get through. So, this is one of the most serious barriers that you have to cross is how to handle security tokens. We'll be going into Kerberos to give you an example of why these security tokens are needed to understand, what are the attackers doing. What are they likely to be doing? We'll be talking about PKI (public key infrastructure) public and private keys. We'll be showing you how to sign documents so that they can't be altered using your private key. We'll be passing our public key into GitHub, so that GitHub knows that it's us that are encrypting messages that are being sent to them.

So, the fact that they can decrypt the message with our public key tells them that it was me, for example, that sent the message using my private key. So, you need to understand the concepts, the basic concepts of security because they're foundational these days. And we'll be giving an example of Auth 2, which is one of the security standards.

